

TRƯỜNG ĐẠI HỌC FPT



FPT UNIVERSITY

LAB211 – OOP WITH JAVA LAB

**FOOTBALLCLUB AND
PLAYERMANAGEMENT REPORT**

INSTRUCTOR'S NAME: LÊ VÕ MINH THƯ

Submission Day: 16 June 2026

SE203056	Nguyễn Phú Khương
----------	-------------------

Contents

1. Project Overview.....	4
Main Features:.....	4
Data Constraints:	4
2. Class Diagram & UML	6
2.1. Class Diagram	6
Class Relationships:.....	7
2.2. Use Case Diagram	7
3. Project Structure.....	8
4. Class Descriptions	10
4.1. Model Layer	10
Person.java (Abstract Class)	10
Player.java (extends Person).....	10
Club.java	10
Validatable.java (Interface).....	11
4.2. Business Layer	11
ClubsManager.java.....	11
PlayersManager.java	12
4.3. Dispatcher Layer	12
Menu.java.....	12
4.4. Tools Layer	12
Inputter.java	12
5. How to Run the Program	13
5.1. System Requirements	13
5.2. Running with IDE (IntelliJ IDEA)	13
5.3. Running with Command Line	13
5.4. Startup Screen	13
6. Feature Usage Guide	15
6.1. Club Management (Features 1-5).....	15

Feature 1: List all clubs.....	15
Feature 2: Add a new club	15
Feature 3: Search club by ID	16
Feature 4: Update a club by ID	16
Feature 5: List clubs with budget <= input.....	17
6.2. Player Management (Features 6-11)	18
Feature 6: List players sorted by club name, shirt number.....	18
Feature 7: Search players by partial name	18
Feature 8: Add a new player	19
Feature 9: Remove a player by ID	20
Feature 10: Update a player by ID	20
Feature 11: List players by position	20
6.3. System — Save / Load / Quit (Features 12-14).....	22
Feature 12: Save data to files	22
Feature 13: Load data from files	22
Feature 14: Quit.....	22
7. OOP Concepts Applied	23
8. Conclusion	23
8.1. Achievements	23
9.2. Challenges & Solutions	23
APPENDIX: Extension — Topic 32: YouthPlayer	24
A1. Extension Overview.....	24
Test Cases from Topic:	24
A2. System Changes.....	24
New Files:	24
Modified Files:	25
Extended Menu (19 Features):.....	25

1. Project Overview

The "Football Club & Player Management" project (J1.L.P0036) is a console-based system developed in Java using a 3-layer architecture. It provides full CRUD functionality for Clubs and Players, along with search, filter, sort, and file-based data persistence.

Main Features:

#	Feature	Target
1	List all clubs	Club
2	Add a new club	Club
3	Search for a club by ID	Club
4	Update a club by ID	Club
5	List clubs with budget <= input value	Club
6	List players sorted by club name, shirt number	Player
7	Search players by partial name	Player
8	Add a new player	Player
9	Remove a player by ID	Player
10	Update a player by ID	Player
11	List players by position	Player
12	Save data to files	System
13	Load data from files	System
14	Quit	System

Data Constraints:

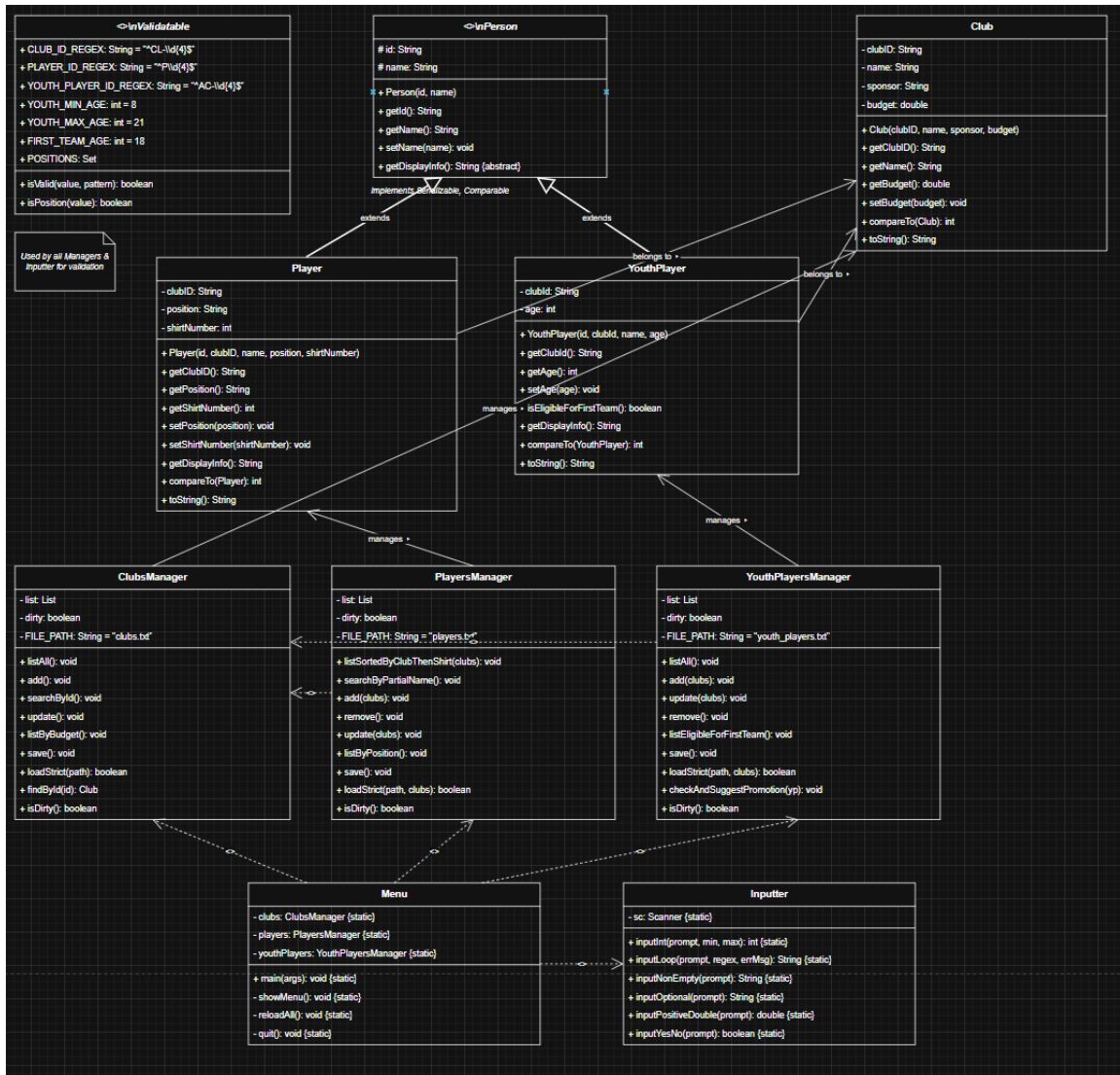
Entity	Attribute	Constraint
Club	clubID	Format: ^CL-\d{4}\$ (CL-0001..CL-9999), unique
Club	name	Non-empty
Club	sponsor	Non-empty
Club	budget	Positive double (million EUR)
Player	id	Format: ^P\d{4}\$ (P0001..P9999), unique
Player	clubID	Must exist in Club list (foreign key)
Player	name	Non-empty
Player	position	Goalkeeper / Defender / Midfielder / Forward / Winger

Player	shirtNumber	1-99, unique within the same Club
--------	-------------	-----------------------------------

2. Class Diagram & UML

2.1. Class Diagram

The Class Diagram below illustrates the relationships between all classes in the core system (excluding the YouthPlayer extension):



Description: Insert the Class Diagram here. Classes: Person (abstract, #id, #name, +getDisplayInfo()), Player (extends Person: -clubID, -position, -shirtNumber), Club (-clubID, -name, -sponsor, -budget), Validatable (interface: constants + isValid(), isPosition()), ClubsManager (List<Club>: CRUD + save/load), PlayersManager (List<Player>: CRUD + save/load), Menu (main: static clubs, players), Inputter (static utility methods). Relationships: Person <|-- Player (extends), Player <-- Club (association via clubID), Menu --> ClubsManager (dependency), Menu --> PlayersManager (dependency).

Class Relationships:

Relationship	From	To	Type
Inheritance	Player	Person	extends
Implementation	Player	Comparable<Player>	implements
Implementation	Club	Comparable<Club>	implements
Implementation	Person	Serializable	implements
Association	PlayersManager	List<Player>	has-a
Association	ClubsManager	List<Club>	has-a
Dependency	Menu	ClubsManager	uses
Dependency	Menu	PlayersManager	uses
Dependency	Menu	Inputter	uses

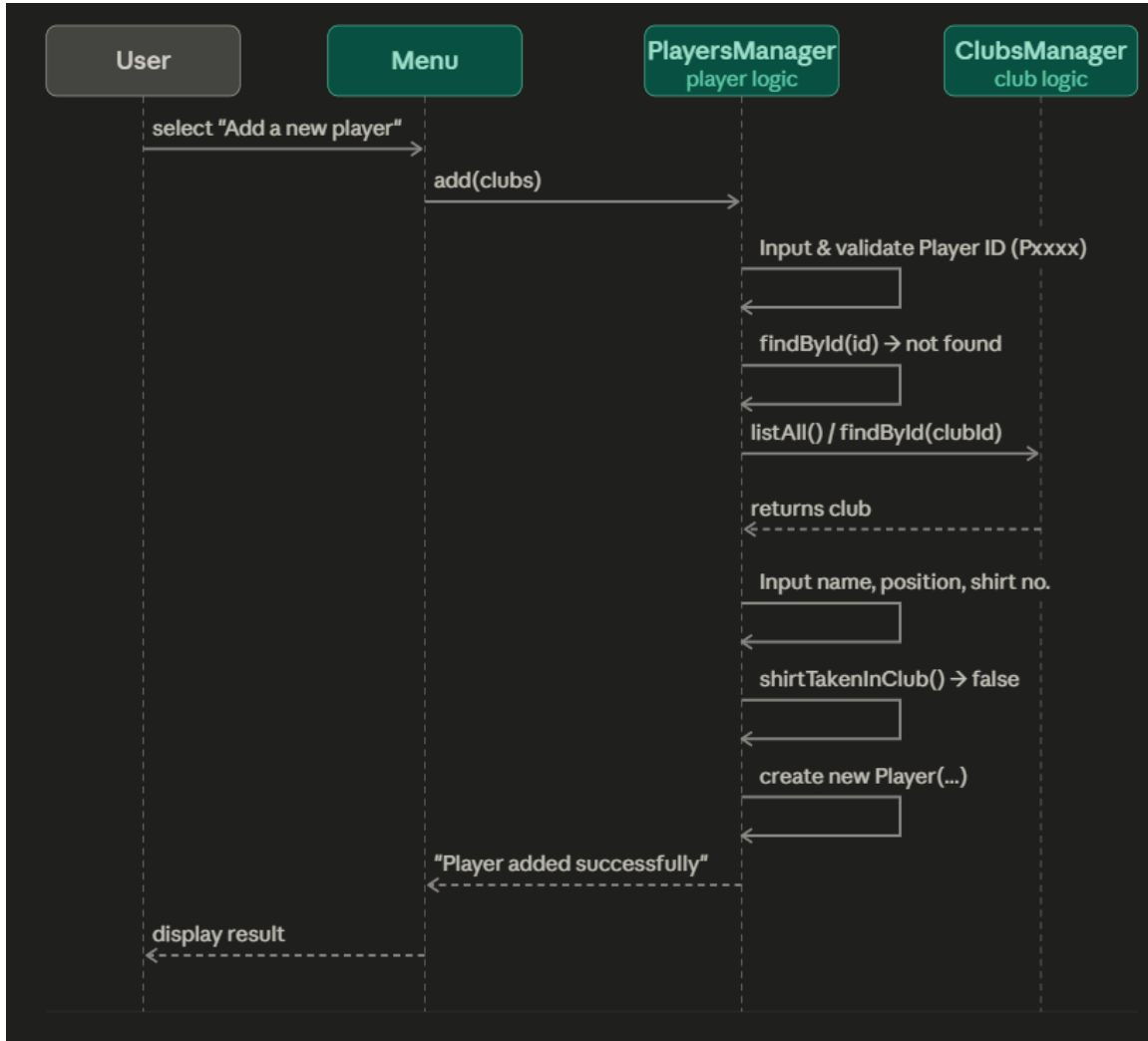
2.2. Use Case Diagram

The Use Case Diagram shows all system functions from the user's perspective:



Description: Insert Use Case Diagram. Actor: User (Manager). Use Cases grouped by: Club (List all, Add, Search by ID, Update, Filter by budget), Player (List sorted, Search by name, Add, Remove, Update, List by position), System (Save, Load, Quit).

2.3. Sequence Diagram



Sequence Diagram

3. Project Structure

The project follows a 3-layer architecture:

```

FootballClub_n_PlayerManagement/
|-- src/
|   |-- model/
|   |   |-- Person.java      <-- Data Layer
|   |   |   (Abstract base class)
|   |   |-- Player.java      (extends Person)
|   |   |-- Club.java         (Serializable, Comparable)
  
```



```
| |   +-- Validatable.java      (Interface - validation constants)
| |   |-- business/            <-- Business Layer
| |       |-- ClubsManager.java (Club CRUD + save/load)
| |       +-- PlayersManager.java (Player CRUD + save/load)
| |   |-- dispatcher/          <-- Presentation Layer
| |       +-- Menu.java         (Main entry point - 14 features)
| +-- tools/                    <-- Utility
|     +-- Inputter.java         (Input validation helper)
|-- clubs.txt                   (Club data file)
+-- players.txt                 (Player data file)
```

The 3-layer architecture ensures clear separation of concerns: Model (data) — Business (logic) — Dispatcher (presentation). The Tools package contains shared utility classes.

4. Class Descriptions

4.1. Model Layer

Person.java (Abstract Class)

Abstract base class representing a person. Contains common attributes (id, name) and defines the abstract method `getDisplayInfo()` for subclass-specific display (Polymorphism).

Member	Type	Description
#id	String (protected)	Unique identifier
#name	String (protected)	Person name
+Person(id, name)	Constructor	Initialize with id and name
+getId()	String	Getter
+getName() / setName()	String	Getter / Setter
+getDisplayInfo()	abstract String	Display info — Polymorphism

Player.java (extends Person)

Represents a football player. Inherits from Person (Inheritance). Adds clubID, position, and shirtNumber. Implements Comparable for natural ordering by shirt number.

Member	Type	Description
-clubID	String	FK -> Club (CL-xxxx)
-position	String	Goalkeeper / Defender / Midfielder / Forward / Winger
-shirtNumber	int	Shirt number (1-99), unique within Club
+getDisplayInfo()	String	Override from Person
+compareTo(Player)	int	Sort by shirt number (Comparable)
+toString()	String	Table-format display
+equals() / hashCode()		Compare by player ID

Club.java

Represents a football club. Implements Serializable and Comparable (sorted by name, case-insensitive).

Member	Type	Description
--------	------	-------------

-clubID	String	Club ID (CL-xxxx)
-name	String	Club name
-sponsor	String	Sponsor brand
-budget	double	Budget (million EUR, > 0)
+compareTo(Club)	int	Sort by name (case-insensitive)
+setBudget(double)	void	Only accepts positive values (Encapsulation)

Validatable.java (Interface)

Interface centralizing all validation rules in one place (Pattern Recognition).

Constant / Method	Value	Description
CLUB_ID_REGEX	^CL-\d{4}\$	Club ID format
PLAYER_ID_REGEX	^P\d{4}\$	Player ID format
POSITIONS	Set<String>	goalkeeper, defender, midfielder, forward, winger
isValid(value, pattern)	boolean	Check if string matches regex
isPosition(value)	boolean	Check if position is valid (case-insensitive)

4.2. Business Layer

ClubsManager.java

Manages CRUD operations for Club. Uses a dirty flag to track unsaved changes.

Method	Function
listAll()	Display all clubs sorted by name (Comparable)
add()	Add new club — validate ID, reject duplicates
searchById()	Search club by ID
update()	Update club — press Enter to keep current value
listByBudget()	Filter clubs with budget <= input value
save()	Write to clubs.txt
loadStrict(path)	Load from file with strict per-line validation
findById(id)	Helper — find club by ID (case-insensitive)

PlayersManager.java

Manages CRUD operations for Player. Uses Comparator chaining for sorting.

Method	Function
listSortedByClubThenShirt(clubs)	Sort by club name then shirt number (Comparator chaining)
searchByPartialName()	Search by name (partial match, case-insensitive)
add(clubs)	Add player — validate ID, clubId, position, unique shirt number
remove()	Remove player by ID
update(clubs)	Update player — press Enter to keep current value
listByPosition()	Filter players by position
save()	Write to players.txt
loadStrict(path, clubs)	Load from file with strict per-line validation

4.3. Dispatcher Layer

Menu.java

Main entry point containing the main() method. Controls the 14-feature menu loop. Auto-loads data on startup and auto-saves on exit when changes are detected (dirty flag).

4.4. Tools Layer

Inputter.java

Utility class for keyboard input with built-in validation:

Method	Description
inputInt(prompt, min, max)	Input integer within [min, max] range
inputLoop(prompt, regex, errMsg)	Input string matching regex pattern
inputNonEmpty(prompt)	Input non-empty string
inputOptional(prompt)	Optional input (Enter to skip)
inputPositiveDouble(prompt)	Input positive double

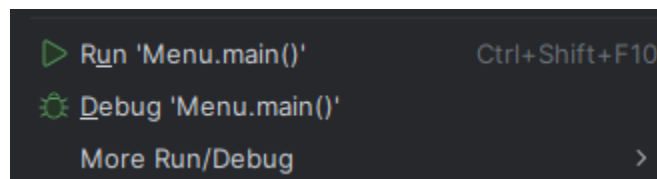
5. How to Run the Program

5.1. System Requirements

- JDK 1.8 or higher (installed and JAVA_HOME configured)
- IDE: IntelliJ IDEA / Eclipse / NetBeans (IntelliJ recommended)
- OS: Windows / macOS / Linux

5.2. Running with IDE (IntelliJ IDEA)

1. Open IntelliJ IDEA -> File -> Open -> select the FootballClub_n_PlayerManagement folder
2. Wait for the IDE to finish indexing all Java files
3. Open file src/dispatcher/Menu.java
4. Right-click on Menu.java -> select "Run 'Menu.main()'"
5. The program will run in the IDE's Console/Terminal window



Description: Screenshot of IntelliJ IDEA right-clicking Menu.java and selecting Run. Show the context menu.

5.3. Running with Command Line

Step 1: Open Terminal and navigate to the project directory.

Step 2: Compile all Java files:

```
javac -encoding UTF-8 -d out -sourcepath src src/dispatcher/Menu.java
```

Step 3: Run the program:

```
java -cp out dispatcher.Menu
```

5.4. Startup Screen

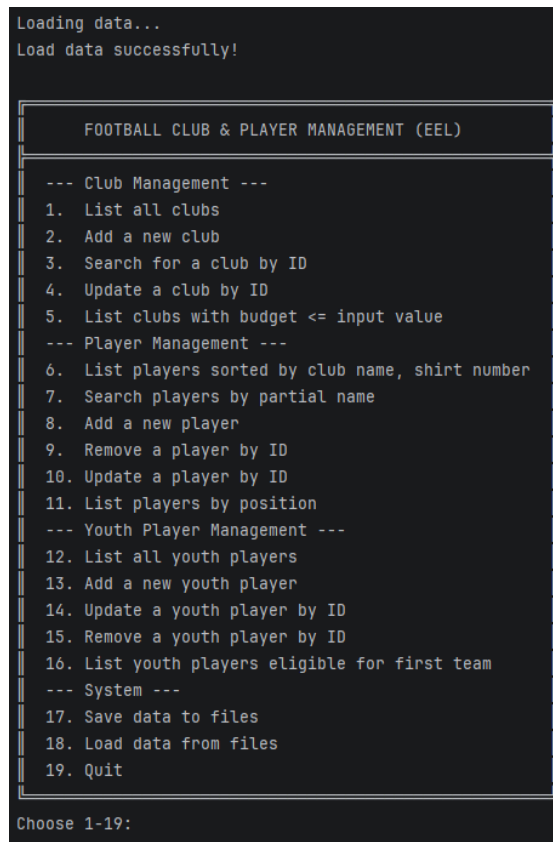
On startup, the system automatically loads data from clubs.txt and players.txt, then displays the main menu with 14 features:

```
=== European Elite League (EEL) Management System ===  
Loading data...
```

Load data successfully!

```
+=====+
|      FOOTBALL CLUB & PLAYER MANAGEMENT (EEL)      |
+=====+
|  1. List all clubs                                  |
|  2. Add a new club                                  |
|  3. Search for a club by ID                         |
|  4. Update a club by ID                             |
|  5. List clubs with budget <= input value           |
|  6. List players sorted by club name, shirt number  |
|  7. Search players by partial name                  |
|  8. Add a new player                                |
|  9. Remove a player by ID                           |
| 10. Update a player by ID                           |
| 11. List players by position                        |
| 12. Save data to files                              |
| 13. Load data from files                           |
| 14. Quit                                             |
+=====+
```

Choose 1-14:



```
Loading data...
Load data successfully!

FOOTBALL CLUB & PLAYER MANAGEMENT (EEL)

--- Club Management ---
1. List all clubs
2. Add a new club
3. Search for a club by ID
4. Update a club by ID
5. List clubs with budget <= input value
--- Player Management ---
6. List players sorted by club name, shirt number
7. Search players by partial name
8. Add a new player
9. Remove a player by ID
10. Update a player by ID
11. List players by position
--- Youth Player Management ---
12. List all youth players
13. Add a new youth player
14. Update a youth player by ID
15. Remove a youth player by ID
16. List youth players eligible for first team
--- System ---
17. Save data to files
18. Load data from files
19. Quit

Choose 1-19:
```

Description: Screenshot of the console displaying "Load data successfully!" and the full 14-feature menu.

6. Feature Usage Guide

6.1. Club Management (Features 1-5)

Feature 1: List all clubs

Enter 1 -> Displays all clubs in table format (Club ID | Club Name | Sponsor | Budget), sorted by club name (Comparable).



```
Choose 1-19: 1
Club ID   | Club Name                | Sponsor  | Budget
-----
CL-0009   | AC Milan                 | Puma     | 610
CL-0010   | AC Milan Sportline      | Puma     | 630
CL-0008   | AS Roma FC              | Adidas   | 720
CL-0004   | Bayern Munich Prime     | Adidas   | 860
CL-0011   | Chelsea FC              | Nike     | 760
CL-0012   | Chelsea FC X Line       | Nike     | 780
CL-0001   | FC Barcelona            | Nike     | 820
CL-0007   | Juventus FC             | Adidas   | 700
CL-0005   | Liverpool FC            | Nike     | 740
CL-0003   | Manchester City Elite   | Puma     | 880
CL-0006   | Paris Saint-Germain     | Nike     | 890
CL-0002   | Real Madrid Galactico   | Adidas   | 910
```

Description: Screenshot of the console displaying the club table with header and data rows.

Feature 2: Add a new club

Enter 2 -> Input the following fields:

6. Club ID (CL-xxxx) — format validated, duplicates rejected
7. Club name — non-empty
8. Sponsor brand — non-empty
9. Budget (million EUR) — must be positive

If all fields are valid -> "Club added successfully."

```
Choose 1-19: 2
Club ID: CL-011
Format must be CL-xxxx
Club ID: CL-0392
Club name: Yu
Sponsor brand: NIKE
Budget (million EUR): 50
Club added successfully.
```

Description: Screenshot of the console adding a new club: all input steps and success message.

Feature 3: Search club by ID

Enter 3 -> Input Club ID -> Displays club info or "This club does not exist!".

```
Choose 1-19: 3
Club ID: CL-0009
Club ID   | Club Name           | Sponsor   | Budget
-----|-----|-----|-----
CL-0009   | AC Milan            | Puma      | 610
```

Description: Screenshot: (a) club found — info displayed, (b) club not found — error message.

Feature 4: Update a club by ID

Enter 4 -> Input Club ID -> Update name, sponsor, budget. Press Enter to skip a field and keep the current value.

```
Choose 1-19: 4
Club ID: CL-0009
Club name [AC Milan] (Enter to skip): CC NiWaki
Sponsor brand [Puma] (Enter to skip): Nike
Budget [610] (Enter to skip): 770
Club updated successfully.
```

Description: Screenshot of updating a club: current values shown, new values entered, success message.

Feature 5: List clubs with budget <= input

Enter 5 -> Input maximum budget -> Displays all clubs with budget <= that value.



Max budget (million EUR): 1000			
Club ID	Club Name	Sponsor	Budget
CL-0009	CC NiWaki	Nike	770
CL-0010	AC Milan Sportline	Puma	630
CL-0008	AS Roma FC	Adidas	720
CL-0004	Bayern Munich Prime	Adidas	860
CL-0011	Chelsea FC	Nike	760
CL-0012	Chelsea FC X Line	Nike	780
CL-0001	FC Barcelona	Nike	820
CL-0007	Juventus FC	Adidas	700
CL-0005	Liverpool FC	Nike	740
CL-0003	Manchester City Elite	Puma	880
CL-0006	Paris Saint-Germain	Nike	890
CL-0002	Real Madrid Galactico	Adidas	910
CL-0392	Yu	NIKE	50

Description: Screenshot: input max budget value, filtered club list displayed.

6.2. Player Management (Features 6-11)

Feature 6: List players sorted by club name, shirt number

Enter 6 -> Displays all players sorted by club name (ascending), then by shirt number within the same club (Comparator chaining).

Choose 1-19: 6

ID	Club ID	Player Name	Position	Shirt
P0010	CL-0004	Joshua Kimmich	Midfielder	6
P0009	CL-0004	Thomas Müller	Forward	25
P0013	CL-0012	Raheem Sterling	Winger	7
P0014	CL-0012	Enzo Fernández	Midfielder	8
P0015	CL-0012	Reece James	Defender	24
P0002	CL-0001	Sergio Busquets	Midfielder	5
P0001	CL-0001	Lionel Messi	Forward	10
P0003	CL-0001	Jordi Alba	Defender	18
P0012	CL-0005	Virgil van Dijk	Defender	4
P0011	CL-0005	Mohamed Salah	Forward	11
P0007	CL-0003	Erling Haaland	Forward	9
P0008	CL-0003	Kevin De Bruyne	Midfielder	17
P0005	CL-0002	Toni Kroos	Midfielder	8
P0004	CL-0002	Luka Modrić	Midfielder	10
P0006	CL-0002	Rodrygo Silva	Forward	21
P2938	CL-0002	Nguyen Phu Khuong	Goalkeeper	33

Description: Screenshot of the player table sorted by club name then shirt number.

Feature 7: Search players by partial name

Enter 7 -> Input a keyword -> Displays all players whose name contains the keyword (case-insensitive).

Choose 1-19: 7

Partial player name: Thomas

ID	Club ID	Player Name	Position	Shirt
P0009	CL-0004	Thomas Müller	Forward	25

Description: Screenshot: input search keyword, matching players displayed.

Feature 8: Add a new player

Enter 8 -> Add a new player with the following steps:

10. Input Player ID (Pxxxx) — format validated, duplicates rejected
11. System displays available clubs for reference
12. Input Club ID (CL-xxxx) — must exist in the club list
13. Input player name — non-empty
14. Input position (Goalkeeper/Defender/Midfielder/Forward/Winger) — validated
15. Input shirt number (1-99) — must be unique within the same club

```
Choose 1-19: 8
Player ID: P0382

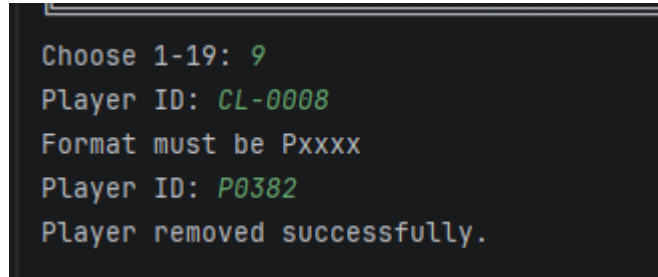
--- Available Clubs ---
Club ID   | Club Name                | Sponsor | Budget
-----
CL-0010   | AC Milan Sportline       | Puma    | 630
CL-0008   | AS Roma FC               | Adidas  | 720
CL-0004   | Bayern Munich Prime      | Adidas  | 860
CL-0009   | CC NiWaki                | Nike    | 770
CL-0011   | Chelsea FC               | Nike    | 760
CL-0012   | Chelsea FC X Line        | Nike    | 780
CL-0001   | FC Barcelona             | Nike    | 820
CL-0007   | Juventus FC              | Adidas  | 700
CL-0005   | Liverpool FC             | Nike    | 740
CL-0003   | Manchester City Elite    | Puma    | 880
CL-0006   | Paris Saint-Germain      | Nike    | 890
CL-0002   | Real Madrid Galactico    | Adidas  | 910
CL-0392   | Yu                       | NIKE    | 50

Club ID: CL-0008
Player name: N P K
Position (Goalkeeper/Defender/Midfielder/Forward/Winger): Forward
Shirt number (1-99): 11
Player added successfully.
```

Description: Screenshot of adding a new player: all input steps and success message.

Feature 9: Remove a player by ID

Enter 9 -> Input Player ID -> Player removed or "This player does not exist!".

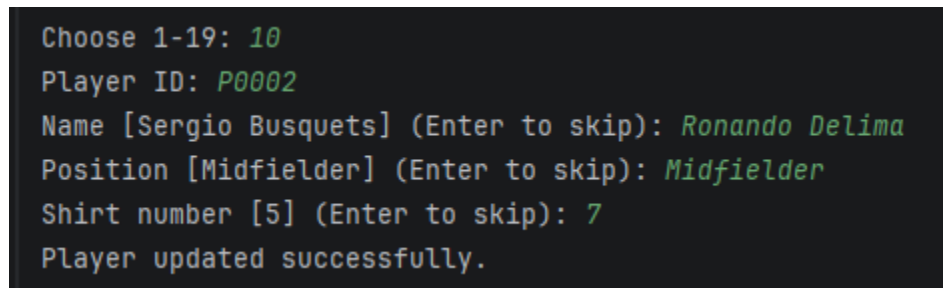


```
Choose 1-19: 9
Player ID: CL-0008
Format must be Pxxxx
Player ID: P0382
Player removed successfully.
```

Description: Screenshot: input player ID, success/error message displayed.

Feature 10: Update a player by ID

Enter 10 -> Input Player ID -> Update name, position, shirt number. Press Enter to skip. Position and shirt number are validated when entered.



```
Choose 1-19: 10
Player ID: P0002
Name [Sergio Busquets] (Enter to skip): Ronando Delima
Position [Midfielder] (Enter to skip): Midfielder
Shirt number [5] (Enter to skip): 7
Player updated successfully.
```

Description: Screenshot: current values shown, new values entered, success message.

Feature 11: List players by position

Enter 11 -> Input a position -> Displays all players in that position.

Choose 1-19: 11

Position: Forward

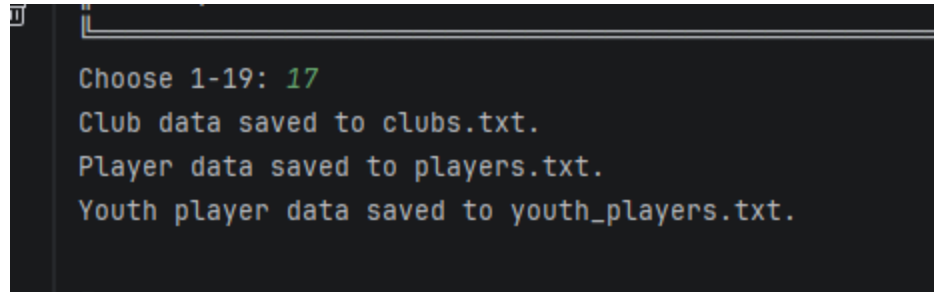
ID	Club ID	Player Name	Position	Shirt
P0001	CL-0001	Lionel Messi	Forward	10
P0006	CL-0002	Rodrygo Silva	Forward	21
P0007	CL-0003	Erling Haaland	Forward	9
P0009	CL-0004	Thomas Müller	Forward	25
P0011	CL-0005	Mohamed Salah	Forward	11

Description: Screenshot: input position name, filtered player list displayed.

6.3. System — Save / Load / Quit (Features 12-14)

Feature 12: Save data to files

Enter 12 -> Saves all Club and Player data to clubs.txt and players.txt respectively.

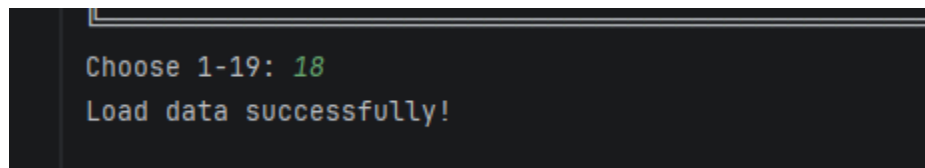


```
Choose 1-19: 17
Club data saved to clubs.txt.
Player data saved to players.txt.
Youth player data saved to youth_players.txt.
```

Description: Screenshot: "Club data saved to clubs.txt." and "Player data saved to players.txt."

Feature 13: Load data from files

Enter 13 -> Reloads all data from files with strict per-line validation. If any line is invalid -> "Load data failed!".

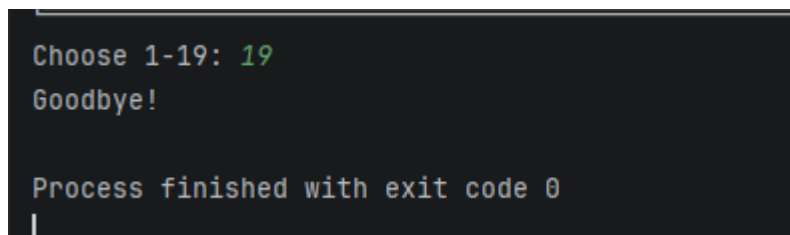


```
Choose 1-19: 18
Load data successfully!
```

Description: Screenshot: (a) successful load, (b) failed load when file is corrupted.

Feature 14: Quit

Enter 14 -> Exits the program. If unsaved changes are detected (dirty flag) -> auto-saves before exiting.



```
Choose 1-19: 19
Goodbye!

Process finished with exit code 0
```

Description: Screenshot: "Changes detected. Saving data before exit..." followed by "Goodbye!"

7. OOP Concepts Applied

OOP Concept	Application	Specific Example
Abstraction	Person is an abstract class defining a common interface	abstract getDisplayInfo() — each subclass defines its own display
Inheritance	Player inherits from Person, reusing id and name	class Player extends Person
Polymorphism	Player overrides getDisplayInfo() and compareTo()	Player.getDisplayInfo() differs from Person's
Encapsulation	Private fields accessed through getter/setter with validation	Club.setBudget() only accepts positive values
Comparable	Club and Player implement Comparable for natural sorting	Club: sort by name; Player: sort by shirt number

8. Conclusion

8.1. Achievements

- Successfully implemented all 14 CRUD features for Club and Player.
- Followed OOP principles: Abstraction, Inheritance, Polymorphism, Encapsulation.
- Strict validation: regex, range, unique constraint, foreign key.
- Flexible sorting: Comparable, Comparator chaining.
- File-based data persistence with strict validation on load.
- Auto-save on exit when changes are detected (dirty flag).
- All 18 test cases PASSED.

9.2. Challenges & Solutions

Challenge	Solution
Sorting players by multiple criteria (club name + shirt)	Used Comparator chaining: <code>byClubName.thenComparingInt()</code>
Validating unique shirt number within the same club	<code>shirtTakenInClub()</code> method iterates list to check duplicates
Strict per-line validation when loading from file	<code>loadStrict()</code> checks field count, format, existence, and duplicates
Unicode encoding on Windows (JDK 1.8)	Used <code>-encoding UTF-8</code> flag during compilation

APPENDIX A :Extension — Topic 32: YouthPlayer

This section describes the YouthPlayer extension feature added to the system as required by Topic 32.

A1. Extension Overview

Topic 32 requires adding a Youth Player management feature. A Youth Player has an ID (AC-xxxx), belongs to a club, has a name and age (8-21). If age ≥ 18 , the system suggests promotion to the first team.

Requirement	Details
Entity	YouthPlayer { id: ^AC-\d{4}\$, clubId, name, age: 8..21 }
Create	Add a youth player to a club
Read	List all youth players & those eligible for first team
Update	Update age; infer first team promotion suggestion
Delete	Delete by ID with confirmation prompt
Validation	age in [8, 21]; age ≥ 18 -> suggest promotion; clubId must exist
OOP	Inherits from Person

Test Cases from Topic:

Input Age	Expected Result	Explanation
19	Valid + Suggest first team promotion	age ≥ 18
7	Rejected	age < 8 (out of range [8, 21])
22	Rejected	age > 21 (out of range [8, 21])

A2. System Changes

New Files:

File	Description
model/YouthPlayer.java	Model extending Person; fields: clubId, age; method: isEligibleForFirstTeam()
business/YouthPlayersManager.java	Full CRUD + promotion suggestion +

	save/load
youth_players.txt	Sample youth player data file

Modified Files:

File	Change
model/Validatable.java	+YOUTH_PLAYER_ID_REGEX, +YOUTH_MIN_AGE (8), +YOUTH_MAX_AGE (21), +FIRST_TEAM_AGE (18)
tools/Inputter.java	+inputYesNo(prompt) — Y/N confirmation method
dispatcher/Menu.java	Menu expanded from 14 to 19 features; added YouthPlayersManager

Extended Menu (19 Features):

```

+=====+
|      FOOTBALL CLUB & PLAYER MANAGEMENT (EEL)      |
+=====+
|  --- Club Management ---                            |
|  1-5   (unchanged)                                |
|  --- Player Management ---                          |
|  6-11  (unchanged)                                |
|  --- Youth Player Management ---                    |
|  12. List all youth players                        * NEW |
|  13. Add a new youth player                        |
|  14. Update a youth player by ID                   |
|  15. Remove a youth player by ID                   |
|  16. List youth players eligible for first team    |
|  --- System ---                                    |
|  17. Save data to files                            |
|  18. Load data from files                          |
|  19. Quit                                           |
+=====+

```

APPENDIX B: Authorization & Access Control (RBAC)

B1. Overview

This extension adds an authorization layer using the Role-Based Access Control (RBAC) model. Each user logs in with a username and password and is assigned a Role. A Role owns a set of Permissions, and every menu feature declares the Permission it requires. The menu then displays only the features the logged-in user is allowed to perform, so unauthorized actions are never even shown.

The design favours OOP principles: permissions are an enum (type-safe), a role aggregates permissions through composition, and the menu is data-driven via MenuItem objects rather than a hard-coded switch. Adding a feature or a role does not require touching the dispatch logic.

B2. Roles & Permissions Matrix

Three roles are provided with the following permissions:

Permission	ADMIN	MANAGER	VIEWER
CLUB_VIEW	Yes	Yes	Yes
CLUB_MANAGE	Yes	No	No
PLAYER_VIEW	Yes	Yes	Yes
PLAYER_MANAGE	Yes	Yes	No
YOUTH_VIEW	Yes	Yes	Yes
YOUTH_MANAGE	Yes	Yes	No
DATA_PERSIST	Yes	Yes	No

Default demo accounts:

Username	Password	Role	Access
admin	admin123	ADMIN	All 18 features
manager	manager123	MANAGER	View clubs; manage players & youth; save/load
viewer	viewer123	VIEWER	Read-only: list/search only

B3. New & Modified Files

File	Type	Description
model/Permission.java	NEW	enum of atomic permissions
model/Role.java	NEW	enum mapping each role to an EnumSet<Permission>; can()
model/User.java	NEW	account holding username/password/role; can(), matches()
business/AuthManager.java	NEW	user store + login() with up to 3 attempts
dispatcher/MenuItem.java	NEW	label + required Permission + Runnable action
dispatcher/Menu.java	MODIFIED	login first, dynamic menu filtered by permission

B4. Key Design Points

- Permission is an enum, giving type-safe, self-documenting capability names.
- Role stores permissions in an EnumSet and exposes can(Permission) - the single source of truth for access checks.

- User delegates can() to its Role, so callers never inspect role internals.
- MenuItem couples each feature with the permission it needs, enabling a uniform filter.
- Menu computes the allowed items once per session and renders a compact, renumbered menu.
- Because unauthorized items are hidden, there is no scattered "Access denied" handling.

B5. Sample Menus per Role

VIEWER sees read-only features only:

```
===== FOOTBALL CLUB & PLAYER MANAGEMENT (EEL) =====
Logged in as: viewer [VIEWER]
-----
1. List all clubs
2. Search for a club by ID
3. List clubs with budget <= input value
4. List players sorted by club name, shirt number
5. Search players by partial name
6. List players by position
7. List all youth players
8. List youth players eligible for first team
9. Quit
```

ADMIN sees the full menu (all 18 features + Quit).